

**POKHARA UNIVERSITY  
GANDAKI COLLEGE OF ENGINEERING AND  
SCIENCE**

**LABORATORY REPORT**



**AGILE SOFTWARE DEVELOPMENT LAB**

Lab No. 3

BESE III YEAR - VI SEM

**Submitted By:**

*Sudip Sharma*

Roll No: 39

Program: BESE ,2023

**Submitted To:**

Er. Rajendra Bahadur Thapa

Lecturer

**Date:** July 3, 2026

# NAME OF EXPERIMENT

Implementation of Different Types of Software Testing Using the Jest Framework.

## AIM

To implement and perform unit testing on a JavaScript calculator application using the Jest framework and to study different software testing tools and techniques such as Test Case Generation, JUnit, Selenium, and Load Testing.

## THEORY

**Software Testing** is a critical phase in the software development lifecycle that involves verifying and validating that a software application functions correctly, meets the specified requirements, and is free from defects. The primary goal of testing is to identify bugs early, ensure reliability, and enhance the overall quality of the product before it is released to end-users.

In modern Agile development, testing is not an afterthought but an integral part of the development process. Different types of testing are performed at various stages, including unit testing, integration testing, system testing, and acceptance testing. This experiment focuses on **unit testing** using the Jest framework, while also surveying other important testing tools.

1. **Test Case Generation:** Test case generation is the systematic process of creating a set of input conditions, execution steps, and expected outputs to verify whether a specific feature of the software behaves as intended. A well-designed test case includes:

- *Test Input:* The data provided to the function.
- *Expected Output:* The correct result that the function should return.
- *Actual Result:* The output produced by the program during execution.

Proper test case generation helps ensure maximum code coverage and helps identify edge cases (such as division by zero or negative inputs) that might otherwise be overlooked.

2. **Jest Framework:** Jest is an open-source JavaScript testing framework developed by Meta (formerly Facebook). It is widely adopted for testing JavaScript applications, particularly those built with React, Node.js, and other modern JavaScript ecosystems. Jest is favored for its simplicity, speed, and rich feature set, which includes:

- *Zero Configuration*: Works out of the box for most JavaScript projects.
  - *Automatic Test Execution*: Watches for file changes and re-runs tests automatically.
  - *Built-in Assertions*: Provides intuitive assertion methods like `expect().toBe()` for verifying outcomes.
  - *Code Coverage Reports*: Generates detailed reports showing which parts of the code are covered by tests.
  - *Snapshot Testing*: Allows developers to capture and compare the state of UI components.
3. **JUnit**: JUnit is a widely used unit testing framework specifically designed for Java applications. It relies on annotations (such as `@Test`, `@Before`, `@After`) to define test cases and setup methods. JUnit integrates seamlessly with build tools like Maven and Gradle and supports automated test execution, making it a cornerstone of Java development.
  4. **Selenium**: Selenium is a powerful open-source automation tool used for testing web applications across different browsers and platforms. Unlike unit testing, which tests individual functions, Selenium automates browser interactions—such as clicking buttons, filling out forms, navigating pages, and validating rendered content. It supports multiple programming languages including Java, Python, C#, and JavaScript, and is essential for end-to-end functional testing.
  5. **Load Testing**: Load testing is a non-functional testing technique that evaluates how a software application performs under varying levels of user traffic and data volume. The goal is to identify performance bottlenecks, measure response times, and ensure the system remains stable under heavy load. Common load testing tools include:
    - *Apache JMeter*: A Java-based tool for performance and load testing.
    - *Artillery*: A modern, scriptable load testing tool for Node.js applications.
    - *k6*: An open-source tool for testing API performance.
    - *Locust*: A Python-based load testing framework.

Load testing is critical for ensuring that applications can handle real-world usage scenarios without crashing or becoming unresponsive.

# OBJECTIVES

- To understand the fundamentals of software testing and its importance in development.
- To implement unit testing for a JavaScript application using the Jest framework.
- To generate effective test cases for arithmetic functions (addition, multiplication, division).
- To study the features and usage of JUnit for Java unit testing.
- To understand how Selenium automates browser-based web application testing.
- To learn the importance of load testing and explore popular load testing tools.

# ALGORITHM

1. Install Node.js and npm (Node Package Manager) on the system.
2. Create a new project directory and initialize it using `npm init -y`.
3. Install Jest as a development dependency using the command: `npm install --save-dev jest`.
4. Create a source file named `calculator.js` and implement three functions: `add()`, `multiply()`, and `divide()`.
5. Export the functions using `module.exports` so they can be imported by the test file.
6. Create a test file named `calculator.test.js` in the same directory.
7. Import the calculator functions into the test file using `require()`.
8. Write individual test cases for each function using Jest's `test()` method and `expect().toBe()` assertions.
9. Include an edge-case test for division by zero to verify that the function throws an appropriate error.
10. Update the `package.json` file to add a test script: `"test": "jest"`.
11. Execute all test cases by running `npm test` or `npx jest` in the terminal.
12. Observe the test results and verify that all test cases pass.

13. Study the theoretical concepts of JUnit, Selenium, and Load Testing as auxiliary testing techniques.

## CODE

### calculator.js

```
1 function add(a, b) {  
2     return a + b;  
3 }  
4  
5 function multiply(a, b) {  
6     return a * b;  
7 }  
8  
9 function divide(a, b) {  
10     if (b === 0) {  
11         throw new Error("Cannot divide by zero");  
12     }  
13     return a / b;  
14 }  
15  
16 module.exports = { add, multiply, divide };
```

## calculator.test.js

```
1 const { add, multiply, divide } = require('./calculator');
2
3 test('Addition Test', () => {
4     expect(add(10, 5)).toBe(15);
5 });
6
7 test('Multiplication Test', () => {
8     expect(multiply(4, 5)).toBe(20);
9 });
10
11 test('Division Test', () => {
12     expect(divide(20, 5)).toBe(4);
13 });
14
15 test('Division by Zero Test', () => {
16     expect(() => divide(20, 0)).toThrow("Cannot divide by zero");
17 });
```

## Command to Run Tests

```
1 npm install --save-dev jest
2 npx jest
```

## TEST CASES

| Test Case        | Input | Expected Output | Result |
|------------------|-------|-----------------|--------|
| Addition         | 10, 5 | 15              | Pass   |
| Multiplication   | 4, 5  | 20              | Pass   |
| Division         | 20, 5 | 4               | Pass   |
| Division by Zero | 20, 0 | Error (Throw)   | Pass   |

Table 1: Test case summary for calculator functions.

## OUTPUT

The calculator application was tested successfully using the Jest framework. The following observations were made:

- The addition test passed successfully ( $10 + 5 = 15$ ).
- The multiplication test passed successfully ( $4 \times 5 = 20$ ).
- The division test passed successfully ( $20 \div 5 = 4$ ).
- The division by zero test successfully threw the expected error message.
- All test cases executed without any failures, confirming the correctness of the implemented functions.
- Jest provided a clear summary of the test run, including the number of passed and failed tests.

## RESULT

The calculator application was successfully unit-tested using the Jest framework. All four test cases, including the edge case for division by zero, passed successfully, demonstrating the correctness and robustness of the implemented arithmetic functions. Additionally, theoretical knowledge about JUnit, Selenium, and Load Testing tools was acquired, broadening the understanding of different testing paradigms used in the software industry.

## CONCLUSION

This experiment provided practical, hands-on experience in software testing using the Jest framework. Unit testing proved to be an effective approach for verifying the correctness of individual functions, catching potential errors early in the development cycle. The study of JUnit, Selenium, and Load Testing highlighted the diversity of testing tools available for different contexts—ranging from back-end unit testing (JUnit) to front-end browser automation (Selenium) and performance evaluation (Load Testing). Embracing a comprehensive testing strategy is essential for delivering high-quality, reliable, and maintainable software in an Agile environment.